CS340 - Analysis of Algorithms

Approximation Algorithms

Warmup

- Complete / Review the last question on the lab

Problem 3. You have 3 machines and jobs with processing times $[2, 3, 4, 6, 2, 2]$.

(a) Run the following greedy algorithm: assign each job (in the given order) to the currently least-loaded machine. What is the makespan?

(b) Find an optimal assignment. What is the optimal makespan?

(c) Construct an instance where greedy does worse relative to optimal.

(d) For parts (a) and (c), compute the ratio (greedy makespan)/(optimal makespan).

Announcements:

HW9 Due today

Exam practice questions posted

Office Hour Change:

- Friday cancelled
- Thursday 2-5 instead (Shared with data structures. Priority early or later?)

No lab Thursday

Do your course reviews

Final Exam Options:

1. Dec 12 1-4pm Park 230
2. Dec 15 9:30-12:30 Park 159

Agenda

1. Load Balancing Review
2. Dealing with NP Complete problems
3. Final Exam Practice Questions

Load Balancing Review

- Given m identical machines and n jobs, each job j has processing time t_j
  - job j must run contiguously on one machine
  - a machine can process at most one job at a time
- Let $J(i)$ be the subset of jobs assigned to machine i. The *load* of machine i is

$$L_i = \sum_{j \in J(i)} t_j$$

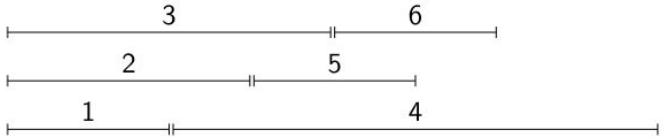
The *makespan* is the maximum load on any machine

$$T = \max_{i=1,..,m} L_i$$

Goal: Can we find a schedule with a *makespan* of at most k?

Jobs	1	2	3	4	5	6
t_i	2	3	4	6	2	2

Consider the following schedule on 3 machines



Approximation Algorithms

Load balancing in polynomial time is likely unattainable...

Can we come up with an algorithm that can runs in polynomial time and finds solutions that are guaranteed to be close to optimal?

Greedy Load Balancing

- Greedy criteria: add jobs to the machine i with the lowest load L_i

- How close is this greedy solution to the optimal solution?
  a. Optimal (T*) = 7
  b. Greedy (T) = 8
  c. For part d, what was the worst ratio you found?

- What is the worst ratio (T/T*) on any given example?
  a. 2

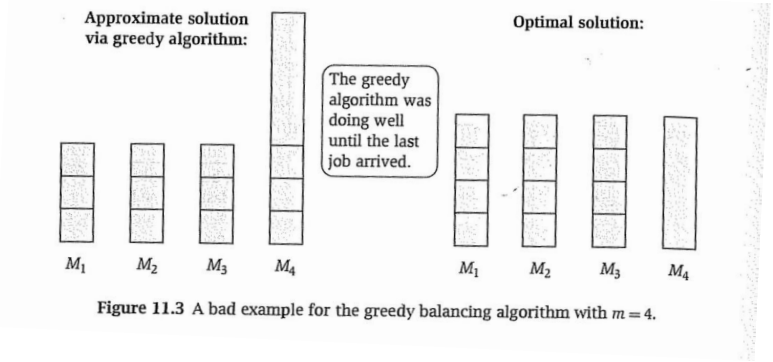


Figure 11.3 A bad example for the greedy balancing algorithm with $m = 4$.

Greedy Load Balancing

What do we know about the optimal makespan T* ?

1. Optimal makespan is at least the total processing time divided by the number of machines

$$L^* \geq \frac{1}{m} \sum_j t_j$$

2. Optimal makespan is at least as large as the longest job

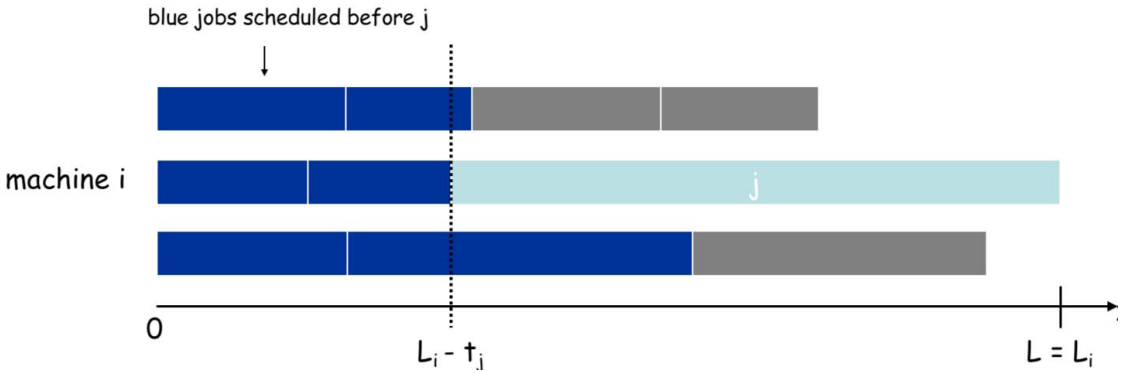
$$L^* \geq \max_j t_j$$

Greedy Load Balancing

What do we know about the greedy makespan T?

1. When each job j is scheduled, it is added to the machine i with the smallest load

$$L_i - t_j \leq L_k \; \forall 1 \leq k \leq m$$



Greedy Load Balancing is a 2-approximation

Claim: For any load balancing input, $T \leq 2T^*$

Proof: $L_i - t_j \leq L_k$

$L_k \leq 1/m \sum_k L_k$

$L_i - t_j \leq 1/m \sum_k L_k$

$L_i - t_j \leq T^*$

$L_i = L_i - t_j + t_j$

$L_i \leq T^* + T^* \leq 2T^*$

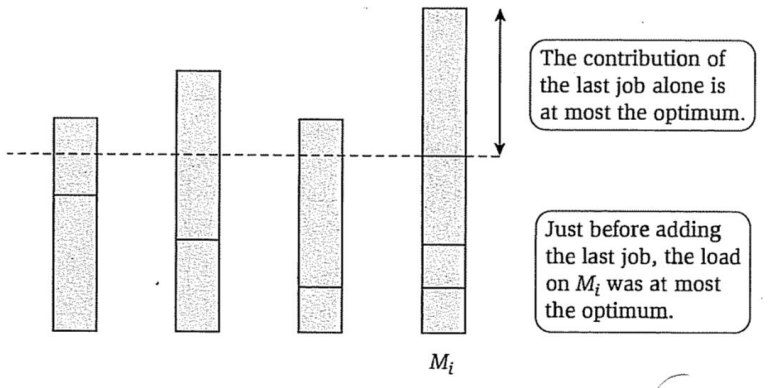


Figure 11.2 Accounting for the load on machine M_i in two parts: the last job to be added, and all the others.

Exam practice problems

Approximation Cautionary Example

Worksheet posted

If A reduces to B, does a good approximation for B guarantee a good approximation for A?

Consider A = VC, B = IS

Greedy criteria: While uncovered edges remain, pick any edge (u,v) and add both u and v to the cover

1. Show that this is a 2-approx of VC
2. Show that this can be **much** worse than a 2-approx on IS G'

Summary

- NP-Complete problems likely can’t be solved in polynomial time
  - We can analyze how close our polynomial time approximation is

- Exam practice problems posted

- **HW9 due today**